

Semana 2 — Access

Objetivo

Implementar el módulo **Access** para gestionar:

- Login administrativo
- Logout
- Usuarios
- Roles
- Permisos
- Asignación de roles a usuarios
- Asignación de permisos a roles
- Middleware de autenticación
- Middleware de autorización por permisos
- Usuario administrador inicial
- Auditoría básica de eventos sensibles

El módulo debe seguir el mismo patrón de Catalog:

```
app/Access/  
├── Domain/  
├── Application/  
├── Infrastructure/  
└── Presentation/
```

1. Decisión importante sobre users

Actualmente ya tienes la tabla users, password_reset_tokens y sessions en la migración base. También tienes un modelo User con password oculto y casteo hashed, lo cual debes conservar.

Como estás en desarrollo, tienes dos opciones: elegí esta opción

Opción A — Recomendada si todavía puedes usar migrate:fresh

Editar la migración existente de users y agregar:

```
is_active  
last_login_at  
last_login_ip  
password_changed_at
```

2. Estructura del módulo Access

```
app/Access/  
├── Domain/  
│   ├── Entities/  
│   │   ├── AccessUser.php  
│   │   ├── Role.php  
│   │   └── Permission.php  
│   └── ValueObjects/  
│       ├── EmailAddress.php  
│       ├── PlainPassword.php  
│       ├── RoleSlug.php  
│       └── PermissionSlug.php
```

- Repositories/
 - UserRepositoryInterface.php
 - RoleRepositoryInterface.php
 - PermissionRepositoryInterface.php
 - AuditLogRepositoryInterface.php

- Services/
 - PermissionChecker.php

- Application/

- DTOs/
 - LoginData.php
 - CreateUserData.php
 - UpdateUserData.php
 - CreateRoleData.php
 - UpdateRoleData.php
 - AssignRolesData.php
 - AssignPermissionsData.php

- UseCases/
 - LoginUseCase.php
 - LogoutUseCase.php
 - ListUsersUseCase.php
 - CreateUserUseCase.php
 - UpdateUserUseCase.php
 - DeleteUserUseCase.php
 - ListRolesUseCase.php
 - CreateRoleUseCase.php
 - UpdateRoleUseCase.php
 - DeleteRoleUseCase.php
 - AssignRolesToUserUseCase.php
 - ListPermissionsUseCase.php
 - AssignPermissionsToRoleUseCase.php

- Infrastructure/

- Models/
 - UserModel.php
 - RoleModel.php
 - PermissionModel.php
 - AccessAuditLogModel.php

- Repositories/
 - EloquentUserRepository.php
 - EloquentRoleRepository.php
 - EloquentPermissionRepository.php
 - EloquentAuditLogRepository.php

- Middleware/
 - EnsureUserHasPermission.php

- Database/
 - Seeders/
 - AccessSeeder.php
 - RoleSeeder.php
 - PermissionSeeder.php
 - AdminUserSeeder.php

- Presentation/

- Controllers/
 - AuthController.php
 - DashboardController.php
 - AdminUserController.php
 - AdminRoleController.php

```
├── AdminPermissionController.php
├── Requests/
│   ├── LoginRequest.php
│   ├── StoreUserRequest.php
│   ├── UpdateUserRequest.php
│   ├── StoreRoleRequest.php
│   ├── UpdateRoleRequest.php
│   ├── AssignRolesRequest.php
│   └── AssignPermissionsRequest.php
├── Resources/
│   ├── UserResource.php
│   ├── RoleResource.php
│   └── PermissionResource.php
```

3. Tablas del módulo Access

users

Ya existe, pero debe quedar así:

```
id
name
email
email_verified_at
password
is_active
last_login_at
last_login_ip
password_changed_at
remember_token
created_at
updated_at
```

Reglas:

```
email único
password hasheado
is_active por defecto true
last_login_ip string 45 nullable
password_changed_at nullable
```

roles

```
id
name
slug
description
is_system
created_at
updated_at
```

Reglas:

```
slug único
```

```
is_system por defecto false
```

Ejemplos:

```
super_admin
admin
catalog_manager
inventory_manager
order_manager
```

permissions

```
id
name
slug
module
description
created_at
updated_at
```

Reglas:

```
slug único
module indexado
```

Ejemplos:

```
access.view
users.view
users.create
users.update
users.delete
roles.view
roles.create
roles.update
roles.delete
roles.assign_permissions
permissions.view
```

role_user

```
id
user_id
role_id
created_at
updated_at
```

Reglas

```
user_id foreign key
role_id foreign key
unique user_id + role_id
cascade on delete
```

permission_role

```
id
```

```
role_id
permission_id
created_at
updated_at
```

Reglas

```
role_id foreign key
permission_id foreign key
unique role_id + permission_id
cascade on delete
```

access_audit_logs

```
role_id foreign key
permission_id foreign key
unique role_id + permission_id
cascade on deleteid
user_id nullable
event
ip_address nullable
user_agent nullable
metadata json nullable
created_at
```

Eventos mínimos

```
login_success
login_failed
logout
user_created
user_updated
user_deleted
role_created
role_updated
role_deleted
permissions_assigned
roles_assigned
```

4. Domain

Responsabilidad

El dominio debe representar reglas de negocio puras, sin depender de Laravel, Request, Auth, Eloquent o vistas.

Entidades

```
AccessUser
Role
Permission
```

Value Objects

```
EmailAddress
PlainPassword
RoleSlug
```

PermissionSlug

Reglas de dominio

- Un usuario inactivo no puede autenticarse.
- Un rol de sistema no debe eliminarse.
- Un permiso debe tener formato módulo.acción.
- Un usuario sin permiso explícito no accede.
- super_admin tiene acceso total.

Ejemplo de formato válido para permisos:

```
catalog.view
catalog.create
users.update
roles.assign_permissions
```

Formato inválido:

```
crear_producto
admin
permiso1
```

5. Application

Tu Catalog usa casos de uso simples que reciben DTOs y delegan a repositorios, como CreateProductUseCase. En Access debes repetir ese patrón.

DTOs

```
LoginData
CreateUserData
UpdateUserData
CreateRoleData
UpdateRoleData
AssignRolesData
AssignPermissionsData
```

Los Requests deben tener métodos tipo toData(), igual que tu StoreProductRequest.

UseCases principales

```
LoginUseCase
LogoutUseCase
CreateUserUseCase
UpdateUserUseCase
DeleteUserUseCase
CreateRoleUseCase
UpdateRoleUseCase
DeleteRoleUseCase
AssignRolesToUserUseCase
AssignPermissionsToRoleUseCase
ListPermissionsUseCase
```

Regla importante

No pasar arrays crudos:

```
// Evitar  
$useCase->execute($request->all());
```

Usar DTO:

```
$useCase->execute($request->toData());
```

6. Infrastructure

Modelos

```
UserModel  
RoleModel  
PermissionModel  
AccessAuditLogModel
```

Aquí hay una decisión técnica importante.

Como Laravel ya usa App\Models\User, puedes hacer una de estas dos cosas:

Opción recomendada para arquitectura limpia --- esta opción elegí

Crear:

```
app/Access/Infrastructure/Models/UserModel.php
```

Y configurar config/auth.php para que Laravel use:

```
App\Access\Infrastructure\Models\UserModel::class
```

Luego puedes dejar App\Models\User como compatibilidad o eliminarlo si no se usa.

7. Repositories

Crear interfaces en Domain:

```
UserRepositoryInterface  
RoleRepositoryInterface  
PermissionRepositoryInterface  
AuditLogRepositoryInterface
```

Crear implementaciones Eloquent:

```
EloquentUserRepository  
EloquentRoleRepository  
EloquentPermissionRepository  
EloquentAuditLogRepository
```

Registrar bindings en AppServiceProvider, igual que ya haces con ProductRepositoryInterface y EloquentProductRepository.

Ejemplo conceptual:

```
$this->app->bind(UserRepositoryInterface::class, EloquentUserRepository::class);  
$this->app->bind(RoleRepositoryInterface::class, EloquentRoleRepository::class);  
$this->app->bind(PermissionRepositoryInterface::class,  
EloquentPermissionRepository::class);  
$this->app->bind(AuditLogRepositoryInterface::class,  
EloquentAuditLogRepository::class);
```

8. Presentation

Controllers

```
AuthController  
DashboardController  
AdminUserController  
AdminRoleController  
AdminPermissionController
```

Requests

```
LoginRequest  
StoreUserRequest  
UpdateUserRequest  
StoreRoleRequest  
UpdateRoleRequest  
AssignRolesRequest  
AssignPermissionsRequest
```

Reglas de validación recomendadas

LoginRequest

```
email: required, email  
password: required, string
```

Además:

- Aplicar rate limiting.
- No revelar si el email existe.
- Mensaje genérico para credenciales incorrectas.

StoreUserRequest

```
name: required, string, max:150  
email: required, email, max:180, unique:users,email  
password: required, string, min:12, confirmed  
is_active: nullable, boolean  
role_ids: nullable, array  
role_ids.*: integer, exists:roles,id
```

UpdateUserRequest

```
name: required, string, max:150  
email: required, email, max:180, unique ignorando usuario actual  
password: nullable, string, min:12, confirmed  
is_active: nullable, boolean  
role_ids: nullable, array  
role_ids.*: integer, exists:roles,id
```


StoreRoleRequest

```
name: required, string, max:120
slug: nullable, string, max:150, unique:roles,slug
description: nullable, string, max:500
permission_ids: nullable, array
permission_ids.*: integer, exists:permissions,id
```

AssignPermissionsRequest

```
permission_ids: required, array
permission_ids.*: integer, exists:permissions,id
```

9. Rutas del módulo Access

Agregar en routes/web.php o separar después en un archivo propio si deseas modularizar más.

Rutas públicas de autenticación

```
Route::middleware('guest')->group(function (): void {
    Route::get('/admin/login', [AuthController::class, 'showLoginForm'])->
>name('admin.login');
    Route::post('/admin/login', [AuthController::class, 'login'])->
>name('admin.login.store');
});
```

Rutas protegidas

```
Route::prefix('admin')
->name('admin.')
->middleware('auth')
->group(function (): void {
    Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

    Route::get('/dashboard', [DashboardController::class, 'index'])
        ->name('dashboard');

    Route::prefix('access')
        ->name('access.')
        ->middleware('permission:access.view')
        ->group(function (): void {
            Route::resource('users', AdminUserController::class)->except(['show']);
            Route::resource('roles', AdminRoleController::class)->except(['show']);
            Route::get('permissions', [AdminPermissionController::class, 'index'])
                ->name('permissions.index');
        });
});
```

Todavía no protejas admin/catalog en esta fase. Eso será en la siguiente etapa: **integrar Access con Catalog**. Tus rutas actuales de catálogo admin todavía están libres de middleware de autenticación/autorización.

10. Middleware de permisos

Crear:

```
app/Access/Infrastructure/Middleware/EnsureUserHasPermission.php
```

Responsabilidad:

- Verificar que el usuario esté autenticado.
- Verificar que el usuario esté activo.
- Permitir todo si tiene rol super_admin.
- Verificar si alguno de sus roles tiene el permiso requerido.
- Si no tiene permiso, abortar con 403.

Uso esperado:

```
->middleware('permission:users.create')  
->middleware('permission:roles.assign_permissions')
```

Regla principal:

Deny by default

Es decir:

Si no tiene permiso, no accede.

11. Seeder inicial

Crear:

```
AccessSeeder  
PermissionSeeder  
RoleSeeder  
AdminUserSeeder
```

Permisos Access

```
access.view  
users.view  
users.create  
users.update  
users.delete  
roles.view  
roles.create  
roles.update  
roles.delete  
roles.assign_permissions  
permissions.view
```

Permisos Catalog

Aunque todavía no se integren, conviene crearlos desde Access:

```
catalog.view  
catalog.create  
catalog.update  
catalog.delete  
catalog.manage_categories  
catalog.manage_brands  
catalog.manage_variants  
catalog.manage_images
```

Roles

```
super_admin
admin
catalog_manager
inventory_manager
order_manager
```

Usuario Inicial

```
name: Super Admin
email: heizen@shopy.test
password: heizen123
role: super_admin
is_active: true
```

Importante

- La contraseña debe guardarse hasheada.
- No registrar la contraseña en logs.
- Cambiar esa contraseña si algún día pasa a producción.

12. Controles OWASP ASVS Nivel 2 aplicados a Access

Autenticación

- Login con email y contraseña.
- Mensajes de error genéricos.
- Rate limiting en login.
- Usuario inactivo no puede iniciar sesión.
- Regenerar sesión después del login.
- Invalidar sesión después del logout.

Contraseñas

- Mínimo 8 caracteres.
- Confirmación de contraseña.
- Hash seguro.
- No exponer password.
- No guardar password en logs.

Autorización

- RBAC: User -> Roles -> Permissions.
- Middleware permission.
- Deny by default.
- super_admin con acceso total.

Auditoría

- Registrar login exitoso.
- Registrar login fallido.
- Registrar logout.
- Registrar creación/edición/eliminación de usuarios.
- Registrar creación/edición/eliminación de roles.
- Registrar asignación de permisos.

Protección de datos

- No exponer password.
- No exponer remember_token.
- No exponer session payload.
- No registrar secretos.